NASCAR for CubeSats: Course Ring Relative Motion

Ian M. Faber*
University of Colorado Boulder, Boulder, CO, 80303

The concept of CubeSat racing is revisited and relative equations of motion for race course rings with respect to the race course origin are explored. Relative motion is separately considered without perturbations, with J_2 perturbations, with drag perturbations, and finally with combined J_2 and drag perturbations. Assumptions from previous projects on CubeSat racing are assessed to be insufficient via analysis of the resulting ring motion, and suggestions for future formation control of the rings to mitigate the effect of perturbations on race course geometry are provided.

I. Nomenclature

A	=	Satellite cross-sectional area
$\mathbf{a}_d$	=	Disturbance acceleration vector
C_d	=	Satellite coefficient of drag
e	=	Orbit eccentricity
f	=	Orbit true anomaly
$\dot{f}$	=	Orbit true anomaly rate of change
$\mathbf{h}$	=	Orbit specific angular momentum vector
h	=	Orbit specific angular momentum
H	=	Atmospheric altitude cutoff
$\hat{\mathbf{i}}$	=	Velocity frame unit vector
J_2	=	Planet J_2 gravitational constant
m	=	Satellite mass
μ	=	Planet gravitational parameter
$\mathcal{N}$	=	Inertial frame
$\hat{\mathbf{o}}$	=	Orbit frame unit vector
$\mathcal{O}$	=	Orbit/Hill frame
$[\mathcal{ON}]$	=	DCM to Orbit frame from Inertial frame
$[\mathcal{OV}]$	=	DCM to Orbit frame from Velocity frame
p	=	Orbit semi-latus rectum
ϕ	=	Orbit planetary latitude
R	=	Gravitational potential function
$\omega_{\mathcal{O}/\mathcal{N}}$	=	Angular velocity of frame $\mathcal{O}$ with respect to frame $\mathcal{N}$
$\mathbf{r}$	=	Position vector
${}^{\mathcal{N}}\mathbf{r}$	=	Position vector in the Inertial frame
r	=	Orbit radius
$\dot{r}$	=	Orbit radius rate of change
r_{eq}	=	Equivalent planetary radius
r_0	=	Reference atmospheric radius
$\boldsymbol{\rho}$	=	Relative position vector
ρ	=	Atmospheric density
ρ_0	=	Reference atmospheric density
$\mathcal{V}$	=	Velocity frame
$\mathbf{v}$	=	Velocity vector
v	=	Satellite speed
x	=	Radial relative separation
$\dot{x}$	=	Radial relative speed
$\ddot{x}$	=	Radial relative acceleration
y	=	Along track relative separation
$\dot{y}$	=	Along track relative speed
$\ddot{y}$	=	Along track relative acceleration
z	=	Cross track relative separation
$\dot{z}$	=	Cross track relative speed
$\ddot{z}$	=	Cross track relative acceleration

*Graduate Student, Ann & H.J. Smead Aerospace Engineering Sciences, 3775 Discovery Dr, Boulder, CO 80303, AIAA student (1600652)

II. Introduction

In a previous project, the idea of CubeSat racing was introduced from an optimal control perspective [1]. The idea was simple: Given a randomized course of waypoint rings and a randomized starting position, compute the optimal trajectory between waypoints that minimizes the time taken for a CubeSat to traverse the entire course while also minimizing the distance to the center of each ring. It was found that primer vector theory [2] in context of the linearized Clohessy-Wiltshire Hill (CWH) equations could be leveraged to generate trajectories for each CubeSat and accomplish the goal.

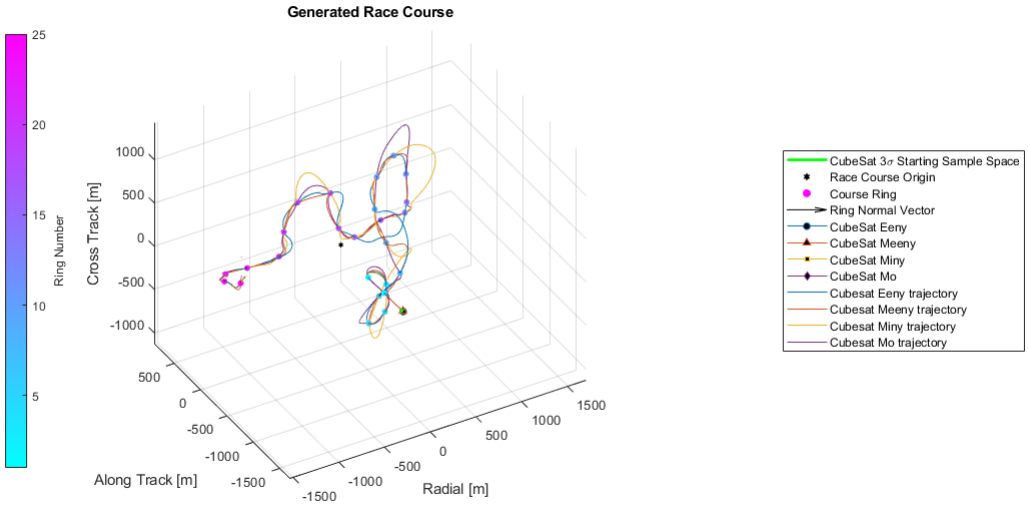

Fig. 1 Example CubeSat Race

However, a critical assumption made was that the rings themselves were not governed by any relative equations of motion, such as the CWH equations. This is not very realistic, as in reality these rings are also in orbit, and there are a number of orbit perturbations present that could change the geometry of the race course over time. This project aims to explore the impact of the full, nonlinear relative equations of motion for spacecraft formations on the race course geometry, as well as to investigate how the two biggest orbit perturbations influence the evolution of the race course over time, namely, J_2 nonuniform gravity and atmospheric drag.

III. Task 1: Randomized Race Course Generation

The first step of this project was to generate CubeSat race courses at random. Luckily, the previous project already accomplished this, and the method is repeated here for context. For this project, improvements were made to how the rings and race course are displayed, and a bug related to how azimuth and elevation angles are converted from relative angles to absolute angles was fixed.

A race course is made up of a sequence of rings that abide by the following requirements:

1) Individual race course rings shall have semi-major and semi-minor axes uniformly distributed between $[1, 5]$ m.
2) Rings shall be spaced apart with uniformly distributed relative azimuth angles $\theta_r \in [-90, 90]^\circ$, relative elevation angles $\phi_r \in [-90, 90]^\circ$, and inter-ring distances $d \in [300, 350]$ m.
    a) The relative azimuth angle is defined relative to the last ring's normal vector in the xy plane, and the relative elevation angle is defined as the angle with respect to the previous ring's normal vector in that ring's z direction.
    b) For example, a relative azimuth and elevation angle of [0, 0]° would point parallel to the last ring. Similarly, an angle combination of [45, -45]° would point to the right and down of the last ring's normal vector.
    c) For plotting, relative azimuth and elevation angles are converted to absolute angles with respect to the race course x axis and race course xy plane, respectively.

3) The first race course ring will be aligned along the course +y axis.
4) Each race course shall consist of 15 rings.

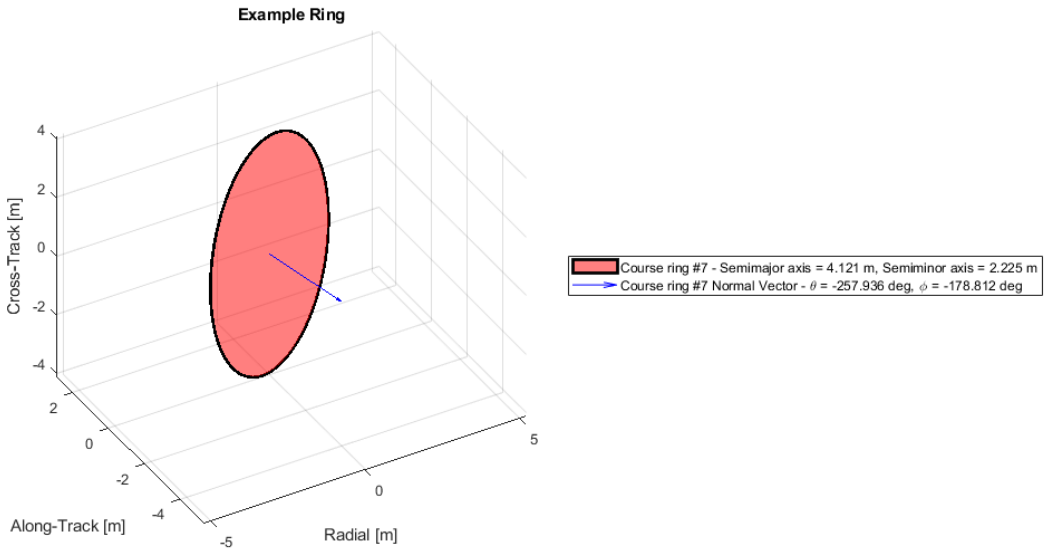

Fig. 2 Example Race Course Ring

Once all rings are generated, the centroid of all the rings is calculated and set as the origin of the race course, which will act as the deputy "spacecraft" for this project. Since it is a point and not a spacecraft by itself, the race course origin does not have perturbations applied to it and will follow a fully Keplerian orbit. This is an assumption that may be relaxed in a follow-on project. Once all rings have been generated and the race course origin assigned, the result is a randomized race course as seen in Figure 3. For the purposes of this project, MATLAB's `rng()` method was given a seed of 3 for repeatability, but the race course can be completely randomly generated with `rng('shuffle')` instead.

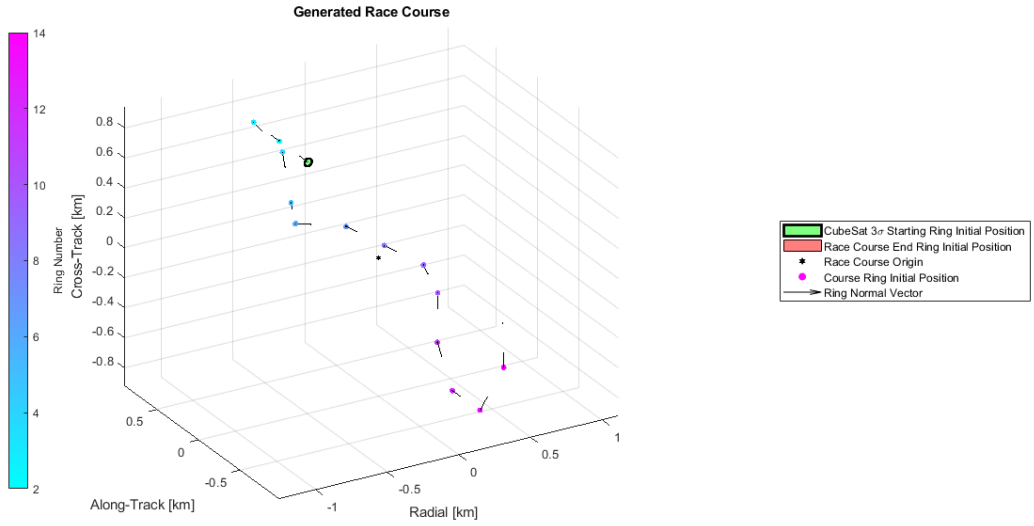

Fig. 3 Example Randomly Generated Race Course

IV. Task 2: Derivation of Ring Relative Equations of Motion, including Perturbations

Next, the relative equations of motion need to be derived for each ring in order to propagate ring relative motion forward in time.

A. Relative EOM Derivation Setup

For a general satellite in orbit about a celestial body, we can write the acceleration vector as follows [3]:

$$\ddot{\mathbf{r}} = -\frac{\mu}{r^3}\mathbf{r} + \mathbf{a}_d, \text{ where} \tag{1}$$

$$\mathbf{a}_d = \mathbf{a}_{J_2} + \mathbf{a}_{drag} \tag{2}$$

We know that the J_2 acceleration can be computed as the gradient of a potential function $R(r)$ as follows:

$$\mathbf{a}_{J_2} = \frac{\partial R(r)}{\partial \mathbf{r}} = \frac{\partial}{\partial \mathbf{r}}\left(\frac{J_2}{2}\frac{\mu}{r}\left(\frac{r_{eq}}{r}\right)^2\left(3\sin^2\phi - 1\right)\right) \tag{3}$$

where $\sin\phi = \hat{u}_r \cdot \hat{u}_z$, $r_{eq} = 6378$ km, and $J_2 = 1.08264\text{e-}3$. Taking the above gradient in inertial coordinates, we get

$$\begin{aligned}
{}^{\mathcal{N}}\mathbf{a}_{J_2} &= \frac{\mu J_2 r_{eq}^2}{r^5}\underbrace{\begin{bmatrix} \frac{15}{2}\sin^2\phi - \frac{3}{2} & 0 & 0 \\ 0 & \frac{15}{2}\sin^2\phi - \frac{3}{2} & 0 \\ 0 & 0 & \frac{15}{2}\sin^2\phi - \frac{9}{2} \end{bmatrix}}_{[A_{J_2}]}{}^{\mathcal{N}}\mathbf{r} \\
{}^{\mathcal{N}}\mathbf{a}_{J_2} &= \frac{\mu J_2 r_{eq}^2}{r^5}\left[A_{J_2}\right]\ {}^{\mathcal{N}}\mathbf{r}
\end{aligned} \tag{4}$$

We also know that the acceleration due to drag can be written as

$$\mathbf{a}_{drag} = -\frac{1}{2}\rho\frac{C_d A}{m}v_{rel}\mathbf{v}_{rel}$$

On orbit, we can approximate $\mathbf{v}_{rel} \approx \mathbf{v}$ as the atmosphere isn't rotating much at the orbit altitude of the satellites. Thus,

$${}^{\mathcal{V}}\mathbf{a}_{drag} = -\frac{1}{2}\rho\frac{C_d A}{m}v\,{}^{\mathcal{V}}\mathbf{v} = -\frac{1}{2}\rho\frac{C_d A}{m}v^2\hat{\mathbf{i}}_v \tag{5}$$

where the atmospheric density is calculated using an exponential atmosphere model like so:

$$\rho(r) = \rho_0 e^{\frac{-(r-r_0)}{H}} \tag{6}$$

As formulated, the J_2 acceleration in Equation 4 is inertial, while the drag acceleration in Equation 5 is in the velocity frame. Generally, relative equations of motion are formulated in the chief's orbit frame, so we need DCMs to the chief's orbit frame from each of the aforementioned frames.

To get to the orbit frame from the inertial frame, we can leverage the following DCM definition:

$$[\mathcal{ON}] = \begin{bmatrix} \left({}^{\mathcal{N}}\hat{\mathbf{o}}_r\right)^\intercal \\ \left({}^{\mathcal{N}}\hat{\mathbf{o}}_\theta\right)^\intercal \\ \left({}^{\mathcal{N}}\hat{\mathbf{o}}_h\right)^\intercal \end{bmatrix} \tag{7}$$

where, if the subscript c denotes quantities derived from the chief, ${}^{\mathcal{N}}\hat{\mathbf{o}}_r = \frac{{}^{\mathcal{N}}\mathbf{r}_c}{r_c}$, ${}^{\mathcal{N}}\hat{\mathbf{o}}_h = \frac{{}^{\mathcal{N}}\mathbf{h}_c}{h_c}$, ${}^{\mathcal{N}}\hat{\mathbf{o}}_\theta = {}^{\mathcal{N}}\hat{\mathbf{o}}_h \times {}^{\mathcal{N}}\hat{\mathbf{o}}_r$

To get the DCM to the orbit frame from the velocity frame, we need to leverage the following definitions:

$$\mathbf{v} = \frac{\mu}{h}\left(e\sin f\hat{\mathbf{o}}_r + \frac{p}{r}\hat{\mathbf{o}}_\theta\right)$$
$$h = \sqrt{\mu p}$$

Then,

$$^{O}\hat{\mathbf{i}}_v = \frac{\mathbf{v}}{v} = \frac{h}{pv}\left(e\sin f\hat{\mathbf{o}}_r + \frac{p}{r}\hat{\mathbf{o}}_\theta\right)$$
$$^{O}\hat{\mathbf{i}}_h = \hat{\mathbf{o}}_h$$
$$^{O}\hat{\mathbf{i}}_n = {}^{O}\hat{\mathbf{i}}_v \times {}^{O}\hat{\mathbf{i}}_h = \frac{h}{pv}\left(\frac{p}{r}\hat{\mathbf{o}}_r - e\sin f\hat{\mathbf{o}}_\theta\right)$$

We can then leverage another DCM definition to get the final DCM to the orbit frame from the velocity frame like so:

$$[\mathcal{OV}] = \begin{bmatrix} {}^{O}\hat{\mathbf{i}}_n & {}^{O}\hat{\mathbf{i}}_v & {}^{O}\hat{\mathbf{i}}_h \end{bmatrix}$$
$$[\mathcal{OV}] = \frac{h}{pv}\begin{bmatrix} \frac{p}{r} & e\sin f & 0 \\ -e\sin f & \frac{p}{r} & 0 \\ 0 & 0 & \frac{pv}{h} \end{bmatrix} \tag{8}$$

B. Relative EOM Derivation

Now, we can begin to derive the relative EOM for a deputy satellite with respect to some chief in the presence of both J_2 and drag orbit perturbations. In this case, the deputy satellite is a course ring, and the chief is the initial centroid of the race course.

The position vector of the deputy can be written as follows in chief orbit frame components:

$$\begin{aligned}\mathbf{r}_d &= \mathbf{r}_c + \boldsymbol{\rho} \\ &= (r_c + x)\hat{\mathbf{o}}_r + y\hat{\mathbf{o}}_\theta + z\hat{\mathbf{o}}_h\end{aligned} \tag{9}$$

Differentiating to get relative velocity,

$$\begin{aligned}\dot{\mathbf{r}}_d &= \frac{{}^{N}\mathrm{d}}{\mathrm{d}t}(\mathbf{r}_d) \\ &= (\dot{r}_c + \dot{x})\,\hat{\mathbf{o}}_r + (r_c + x)\,\dot{\hat{\mathbf{o}}}_r + \dot{y}\hat{\mathbf{o}}_\theta + y\dot{\hat{\mathbf{o}}}_\theta + \dot{z}\hat{\mathbf{o}}_h + z\dot{\hat{\mathbf{o}}}_h\end{aligned}$$

Leveraging transport theorem and assuming $\boldsymbol{\omega}_{O/N} = \dot{f}\hat{\mathbf{o}}_h$ since the chief orbit is Keplerian,

$$\begin{aligned}\dot{\hat{\mathbf{o}}}_r &= \frac{{}^{O}\mathrm{d}}{\mathrm{d}t}(\hat{\mathbf{o}}_r) + \boldsymbol{\omega}_{O/N} \times \hat{\mathbf{o}}_r \\ &= \dot{f}\hat{\mathbf{o}}_\theta \\ \dot{\hat{\mathbf{o}}}_\theta &= \frac{{}^{O}\mathrm{d}}{\mathrm{d}t}(\hat{\mathbf{o}}_\theta) + \boldsymbol{\omega}_{O/N} \times \hat{\mathbf{o}}_\theta \\ &= -\dot{f}\hat{\mathbf{o}}_r \\ \dot{\hat{\mathbf{o}}}_h &= \mathbf{0}\end{aligned}$$

Then, the deputy relative velocity can be expressed as follows:

$$\dot{\mathbf{r}}_d = \left(\dot{r}_c + \dot{x} - y\dot{f}\right)\hat{\mathbf{o}}_r + \left((r_c + x)\,\dot{f} + \dot{y}\right)\hat{\mathbf{o}}_\theta + \dot{z}\hat{\mathbf{o}}_h \tag{10}$$

Differentiating one last time to get relative acceleration and leveraging transport theorem,

$$\begin{aligned}
\ddot{\mathbf{r}}_d &= \frac{\mathcal{N}\mathrm{d}}{\mathrm{d}t}\left(\dot{\mathbf{r}}_d\right) \\
&= \left(\ddot{r}_c + \ddot{x} - \dot{y}\dot{f} - y\ddot{f}\right)\hat{\mathbf{o}}_r + \left(\dot{r}_c + \dot{x} - y\dot{f}\right)\dot{\hat{\mathbf{o}}}_r \\
&\quad + \left(\left(\dot{r}_c + \dot{x}\right)\dot{f} + \left(r_c + x\right)\ddot{f} + \ddot{y}\right)\hat{\mathbf{o}}_\theta + \left(\left(r_c + x\right)\dot{f} + \dot{y}\right)\dot{\hat{\mathbf{o}}}_\theta \\
&\quad + \ddot{z}\hat{\mathbf{o}}_h + \dot{z}\dot{\hat{\mathbf{o}}}_h \\
&= \left(\ddot{r}_c + \ddot{x} - 2\dot{y}\dot{f} - \left(r_c + x\right)\dot{f}^2 - y\ddot{f}\right)\hat{\mathbf{o}}_r \\
&\quad + \left(\ddot{y} + \left(r_c + x\right)\ddot{f} + 2\left(\dot{r}_c + \dot{x}\right)\dot{f} - y\dot{f}^2\right)\hat{\mathbf{o}}_\theta \\
&\quad + \ddot{z}\hat{\mathbf{o}}_h
\end{aligned} \tag{11}$$

We can simplify Equation 11 by utilizing what we know about the chief orbit. Following a similar process as above, we can get the following relationships:

$$\begin{aligned}
\mathbf{r}_c &= r_c\hat{\mathbf{o}}_r \\
\dot{\mathbf{r}}_c &= \dot{r}_c\hat{\mathbf{o}}_r + r_c\dot{f}\hat{\mathbf{o}}_\theta \\
\ddot{\mathbf{r}}_c &= \left(\ddot{r}_c - r_c\dot{f}^2\right)\hat{\mathbf{o}}_r + \left(2\dot{r}_c\dot{f} + r_c\ddot{f}\right)\hat{\mathbf{o}}_\theta
\end{aligned}$$

Since we assumed the chief was on a Keplerian orbit and hence not affected by J_2 or drag perturbations, we also know that $\ddot{\mathbf{r}}_c = -\frac{\mu}{r_c^3}\mathbf{r}_c = -\frac{\mu}{r_c^2}\hat{\mathbf{o}}_r$. Equating,

$$\begin{aligned}
\ddot{r}_c - r_c\dot{f}^2 &= -\frac{\mu}{r_c^2} \\
2\dot{r}_c\dot{f} + r_c\ddot{f} &= 0
\end{aligned}$$

Recalling that $h = \sqrt{\mu p} = r^2\dot{f}$,

$$\begin{aligned}
\ddot{r}_c &= r_c\dot{f}^2\left(1 - \frac{r_c}{p}\right) \\
\ddot{f} &= -2\frac{\dot{r}_c}{r_c}\dot{f}
\end{aligned}$$

Finally, plugging everything in to get the kinematic relative equations of motion,

$$\begin{aligned}
\ddot{\mathbf{r}}_d = &\left(\ddot{x} - 2\dot{f}\left(\dot{y} - \frac{\dot{r}_c}{r_c}y\right) - x\dot{f}^2 - \frac{\mu}{r_c^2}\right)\hat{\mathbf{o}}_r \\
&+ \left(\ddot{y} + 2\dot{f}\left(\dot{x} - \frac{\dot{r}_c}{r_c}x\right) - y\dot{f}^2\right)\hat{\mathbf{o}}_\theta \\
&+ \ddot{z}\hat{\mathbf{o}}_h
\end{aligned} \tag{12}$$

Now, we need to incorporate the dynamics of the deputy. We know that $\ddot{\mathbf{r}}_d = -\frac{\mu}{r_d^3}\mathbf{r}_d + \mathbf{a}_d$. Applying the DCMs from Equations 7 and 8,

$$
\begin{aligned}
-\frac{\mu}{r_d^3}\mathbf{r}_d + \mathbf{a}_d &= [O\mathcal{N}]\left(-\frac{\mu}{r_d^3}{}^{\mathcal{N}}\mathbf{r}_d + \frac{\mu J_2 r_{eq}^2}{r_d^5}\left[A_{J_2}\right]{}^{\mathcal{N}}\mathbf{r}_d\right) \\
&\quad + [O\mathcal{V}_d]\left(-\frac{1}{2}\rho\frac{C_d A_d}{m_d}v_d{}^{\mathcal{V}}\mathbf{v}_d\right) \\
&= -\frac{\mu}{r_d^3}\begin{bmatrix} r_c + x \\ y \\ z \end{bmatrix} + \frac{\mu J_2 r_{eq}^2}{r_d^5}[O\mathcal{N}]\left[A_{J_2}\right]{}^{\mathcal{N}}\mathbf{r}_d - \frac{1}{2}\rho\frac{C_d A_d}{m_d}v_d^2[O\mathcal{V}_d]\hat{\mathbf{i}}_v
\end{aligned} \tag{13}
$$

Here, $[O\mathcal{V}_d]$ needs to map to the chief orbit frame from the deputy velocity frame. We can split this DCM into a product of simpler DCMs that we know how to evaluate:

$$
\begin{aligned}
[O\mathcal{V}_d] &= [OO_d]\,[O_d\mathcal{V}_d] \\
&= [O\mathcal{N}]\,[O_d\mathcal{N}]^{\intercal}\,[O_d\mathcal{V}_d]
\end{aligned}
$$

We know from Equation 8 that

$$
[O_d\mathcal{V}_d] = \frac{h_d}{p_d v_d}\begin{bmatrix} \frac{p_d}{r_d} & e_d\sin f_d & 0 \\ -e_d\sin f_d & \frac{p_d}{r_d} & 0 \\ 0 & 0 & \frac{p_d v_d}{h_d} \end{bmatrix} \tag{14}
$$

Plugging Equation 14 into Equation 13,

$$
\ddot{\mathbf{r}}_d = -\frac{\mu}{r_d^3}\begin{bmatrix} r_c + x \\ y \\ z \end{bmatrix} + \frac{\mu J_2 r_{eq}^2}{r_d^5}[O\mathcal{N}]\left[A_{J_2}\right]{}^{\mathcal{N}}\mathbf{r}_d - \frac{1}{2}\rho\frac{C_d A_d}{m_d}v_d\,[OO_d]\begin{bmatrix} \frac{h_d e_d\sin f_d}{p_d} \\ \frac{h_d}{r_d} \\ 0 \end{bmatrix} \tag{15}
$$

Finally, we can equate Equations 12 and 15 to get the full, nonlinear equations of motion for each ring in the presence of J_2 and drag:

$$
\begin{bmatrix} \ddot{x} - 2\dot{f}\left(\dot{y} - \frac{\dot{r}_c}{r_c}y\right) - x\dot{f}^2 - \frac{\mu}{r_c^2} \\ \ddot{y} + 2\dot{f}\left(\dot{x} - \frac{\dot{r}_c}{r_c}x\right) - y\dot{f}^2 \\ \ddot{z} \end{bmatrix} = -\frac{\mu}{r_d^3}\begin{bmatrix} r_c + x \\ y \\ z \end{bmatrix} + \frac{\mu J_2 r_{eq}^2}{r_d^5}[O\mathcal{N}]\left[A_{J_2}\right]{}^{\mathcal{N}}\mathbf{r}_d - \frac{1}{2}\rho\frac{C_d A_d}{m_d}v_d\,[OO_d]\begin{bmatrix} \frac{h_d e_d\sin f_d}{p_d} \\ \frac{h_d}{r_d} \\ 0 \end{bmatrix} \tag{16}
$$

This is a complicated system to solve analytically, as both the deputy and chief states are changing independently with time. Further, drag is a non-conservative force, which means that any symmetries we could exploit with just gravitational forces are invalidated. In the interest of time, this project focuses on a numerical integration approach. Thus, at each time step we will need to keep track of both the deputy and chief motions in order to correctly calculate the DCMs and disturbance acceleration on the deputy, and hence the relative equations of motion. As will be shown, this comes with the risk of small numerical errors building up with time, particularly with respect to the DCMs.

V. Task 3: Multi-Ring Integration Scheme

In order to get results in a reasonable time, an integration scheme that can handle 15 rings and a chief spacecraft at once is required. This can be accomplished by realizing that the only coupling between spacecraft is between the chief and deputies, as each deputy does not depend on the others for its equations of motion. Thus, we can integrate all of the deputies simultaneously as long as we integrate the chief motion first. In practice, we can do this by constructing a "mega-state" of all spacecraft in the simulation like so:

$$\mathcal{X}_{mega} = \begin{bmatrix} \mathbf{X}_{chief} \\ \mathbf{X}_{deputy,1} \\ \mathbf{X}_{deputy,2} \\ \cdots \\ \mathbf{X}_{deputy,n} \end{bmatrix}$$

where $\mathbf{X}_{chief} = \left[X_c, Y_c, Z_c, \dot{X}_c, \dot{Y}_c, \dot{Z}_c\right]^\intercal$ and $\mathbf{X}_{deputy,i} = [x_i, y_i, z_i, \dot{x}_i, \dot{y}_i, \dot{z}_i]^\intercal$ for $i \in [1, n]$.

Then, we simply propagate the chief motion and loop over all the deputies, solving the system specified in Equation 16 for $\dot{\mathbf{X}}_{deputy,i}$ for each deputy. This can be easily done in MATLAB with a numerical integrator like `ode45`, where we define an outer wrapper function for looping through the deputies and then an inner EOM function that solves Equation 16 for each deputy. The code that accomplishes this can be seen in Appendix B.

VI. Task 4: Course Ring Relative Motion Investigation

We can now investigate how the CubeSat race course geometry might evolve in the presence of relative motion and orbit perturbations. For this investigation, Table 2 describes the race course origin orbit elements, and Table 3 describes the individual initial ring parameters. The previous project found that a typical CubeSat race took about 17 minutes, so all scenarios presented here are propagated for that long. Links to scenario animations can be found in Appendix C.

Table 2 Race Course Origin Orbit Elements

Element	Value
Semi-major axis	8000 km
Eccentricity	0.18
Inclination	45°
RAAN	15°
Argument of periapsis	-23°
Initial true anomaly	10°

Table 3 Race Course Ring Parameters

Ring	Center [km]	Area [km^2]	C_d	Mass [kg]
1	[-0.6733, -0.3162, 0.9219]	7.0676e-4	1.2	100
2	[-0.6733, 0.0338, 0.9219]	2.8274e-5	1.2	10
3	[-0.6733, 0.3442, 0.9219]	7.7907e-6	1.2	10
4	[-0.4617, 0.3613, 0.6660]	3.5798e-6	1.2	10
5	[-0.4320, 0.3093, 0.3436]	4.1689e-6	1.2	10
6	[-0.3022, 0.4818, 0.1046]	9.1055e-6	1.2	10
7	[-0.0176, 0.3618, 0.0672]	1.0373e-5	1.2	10
8	[0.0545, 0.0244, 0.0600]	2.0463e-5	1.2	10
9	[0.3223, 0.0159, -0.1290]	5.5945e-6	1.2	10
10	[0.4751, 0.1067, -0.3872]	8.5411e-6	1.2	10
11	[0.4553, 0.0761, -0.6993]	1.6805e-5	1.2	10
12	[0.4218, -0.1619, -0.9190]	3.7198e-6	1.2	10
13	[0.4209, -0.4992, -0.9126]	1.0609e-5	1.2	10
14	[0.5475, -0.5664, -0.6298]	1.0361e-5	1.2	10
15	[0.5358, -0.5715, -0.3301]	1.1310e-6	1.2	1

A semimajor axis of 8000 km and eccentricity of 0.18 were chosen to verify that the full nonlinear equations of motion were correctly implemented, as well as to provide a large enough altitude change for drag to affect the trajectory of each ring. These elements result in an apoapsis and periapsis of 9440 and 6560 km respectively, corresponding to orders of magnitude changes in atmospheric density. In practice, the race course origin would ideally be in a high enough orbit for drag to be negligible, but as an academic exercise it was desired for drag to have an observable effect. The other elements were more or less chosen randomly, with the exception of inclination and true anomaly in order to more effectively pull out the influence of J_2 and drag respectively.

The ring parameters were dictated mostly by the race course requirements. Ring cross-sectional area is determined by $A = \pi\,(ab - 0.64ab)$ to approximate elliptical annuli. Masses were determined by the type of ring, namely, the larger start ring has the largest mass, medium-sized intermediate rings have intermediate mass, and the smaller end ring has the smallest mass. The coefficient of drag was chosen to be 1.2 based on [4], but is not a particularly rigorous choice as fluid dynamics is not the focus of this report and the author's knowledge of it is subpar. An improvement to this investigation could be a more intelligent selection of ring parameters, but since the focus of this report is on the relative EOM between the course origin and each ring, the parameters chosen were deemed to be sufficient.

A. Ring Relative Motion, no perturbations

The first part of this investigation was to see how the simple inclusion of relative equations of motion would affect race course geometry, as the original project didn't apply relative EOM to the rings. After turning off perturbations, Equation 16 was propagated forward in time for 17 minutes, resulting in Figure 4. The results were also checked with a truth inertial simulation, in which each ring and the chief were integrated in inertial space, then mapped to the chief orbit frame. The differences can be seen in Figure 5.

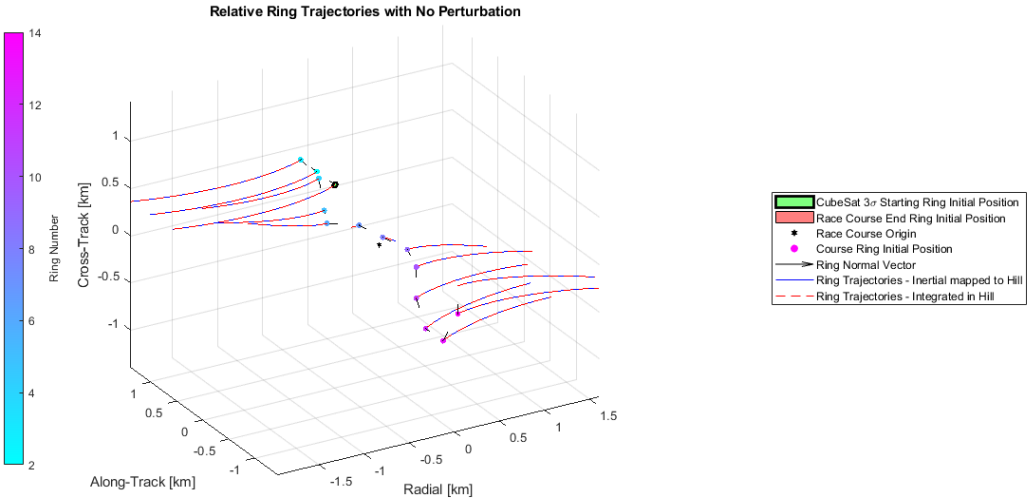

Fig. 4 Race Course Geometry Evolution, no perturbations

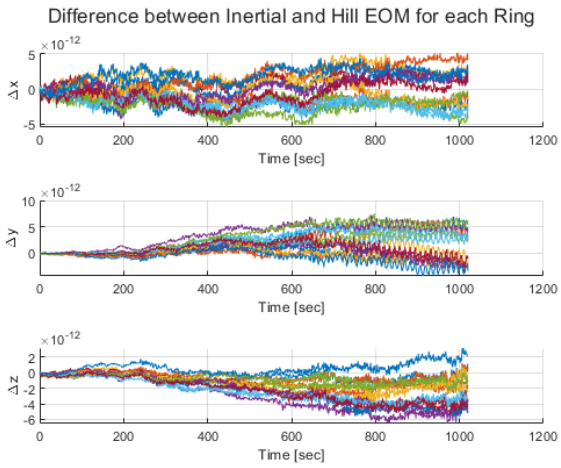

Fig. 5 Difference in Orbit frame and Inertial frame EOM integration, no perturbations

As expected, even in the absence of orbit perturbations the race course geometry shifts noticably over the average duration of a CubeSat race. In particular, Figure 4 shows a gradual "fanning out" of the rings along the chief orbit's inclination, with rings in the negative radial direction migrating closer to Earth and rings in the positive radial direction migrating further away. However, the migrations are on the order of 1.5 km, which isn't anything to get alarmed with yet.

As for solution accuracy, Figure 5 shows differences between the inertially integrated EOM and orbit frame (aka Hill frame) integrated EOM on the order of 1e-12 km, or about 1 nm, which is 400x shorter than the wavelength of visible light. This provides strong evidence that the EOM derived in Section IV.B are correct. However, we do see a small amount of drifting over the period of propagation, which could be attributed to small numerical errors from the numerical integration process. Overall, however, the accuracy appears to be spot-on.

B. Ring Relative Motion, J_2 perturbation only

The next step of the investigation was to turn on J_2 nonuniform gravity, which is arguably one of the biggest orbit perturbations for satellites in LEO. J_2 was turned on and Equation 16 was propagated forward in time for 17 minutes, resulting in Figure 6. Once again, an inertial simulation was used to check the relative results, and the differences are shown in Figure 7.

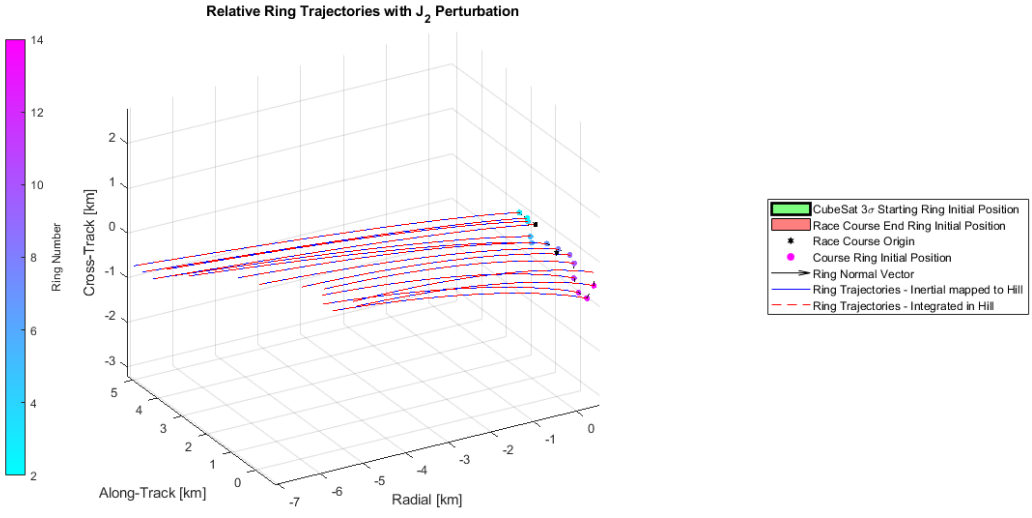

Fig. 6 Race Course Geometry Evolution, J_2 perturbation only

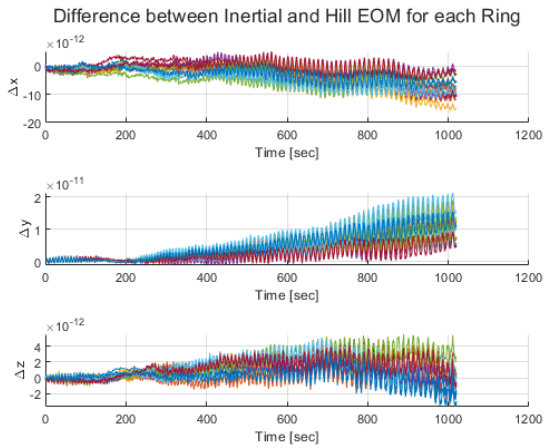

Fig. 7 Difference in Orbit frame and Inertial frame EOM integration, J_2 perturbation only

Figure 6 shows a much more dramatic evolution of the race course geometry, where we can see all the rings migrate closer to the Earth and downwards. This makes sense due to the chief's inclined orbit, which results in an additional acceleration component in the inertial z direction, which pulls the rings in the cross- and along-track directions. Further, there is more acceleration acting on the rings than the chief, which explains why the apparent motion migrates closer to the Earth. While not explored here, it would be interesting to see how egregious the drift is if J_2 was also applied to the chief orbit, as the J_2 acceleration shouldn't vary too much across rings given their proximity to the chief's radius.

Figure 7 shows similarly minuscule differences between the inertial and orbit frame integrations to those in Figure 5, but there is now a high-frequency oscillation. This can be attributed to the application of the time varying $[ON]$ DCM, which introduces small numerical errors that are then integrated. However, the errors are still minuscule and the accuracy is sufficient for this report.

C. Ring Relative Motion, drag perturbation only

Next, J_2 was disabled and drag was turned on. Equation 16 was propagated for 17 minutes, and Figure 8 was generated. An inertial simulation was used to check the results, and the differences are shown in Figure 9.

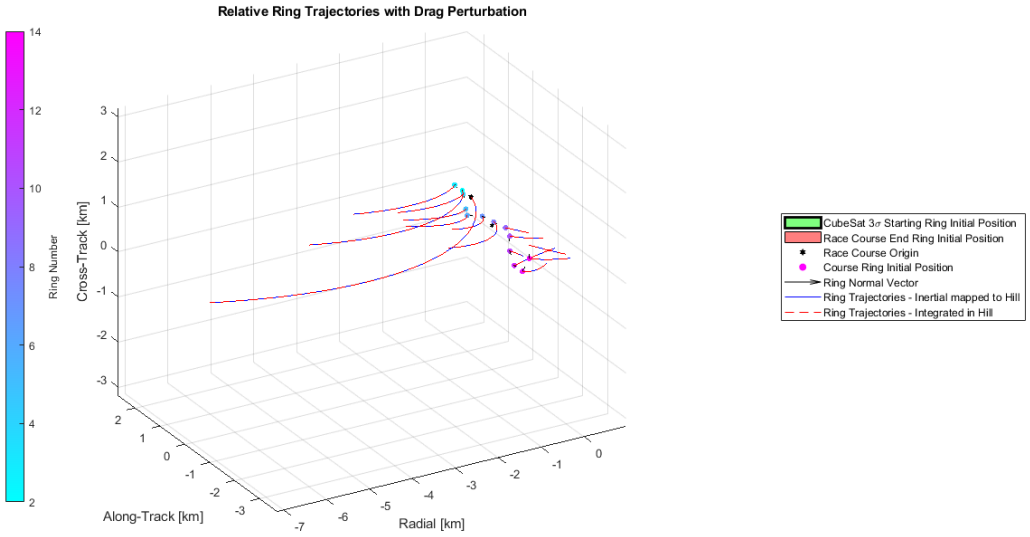

Fig. 8 Race Course Geometry Evolution, drag perturbation only

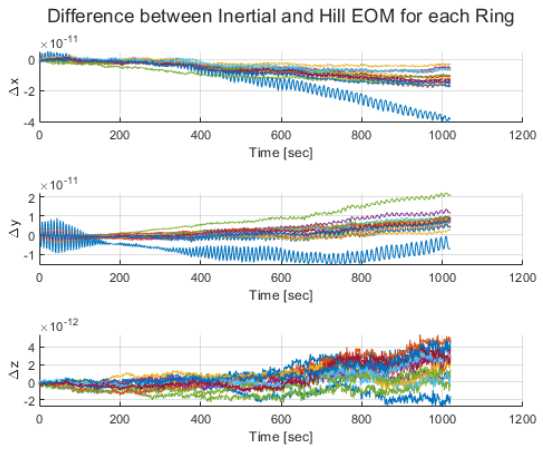

Fig. 9 Difference in Orbit frame and Inertial frame EOM integration, drag perturbation only

Figure 8 shows a similarly dramatic effect on the race course geometry as Figure 6, on the order of 7 km for some rings. In particular, the rings with the largest areas are affected the most, while the rings with the smallest areas are only mildly affected. The effect of drag is exaggerated due to the choice of the chief orbit and conservative choice of ring parameters, but it is a good illustration of what will happen to the race course geometry if a suboptimal chief orbit is chosen. The migration towards Earth makes sense as well, as drag is siphoning away some orbit energy from all the rings and causing them to fall closer to Earth. We can also see the general motion of the rings is away from the chief, which makes sense as all the rings should be slowing down in the presence of drag.

Figure 9 shows similarly small errors to the preceding simulation checks, but with more high frequency oscillations. This once again points to the application of DCMs, in particular the $[OO_d]$ DCM. The errors are still on the order of 1e-11 km, so the accuracy is sufficient.

D. Ring Relative Motion, J_2 + drag combined perturbations

Finally, both J_2 and drag perturbations were enabled, and Equation 16 was propagated for 17 minutes. Figure 10 shows the resulting relative motion, and Figure 11 shows differences with the inertial truth integration.

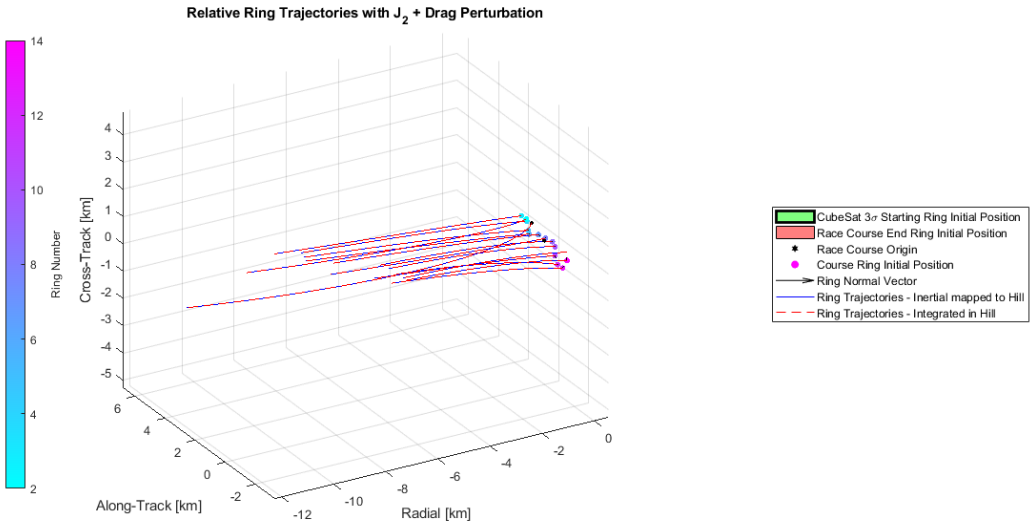

Fig. 10 Race Course Geometry Evolution, J_2 + drag perturbations

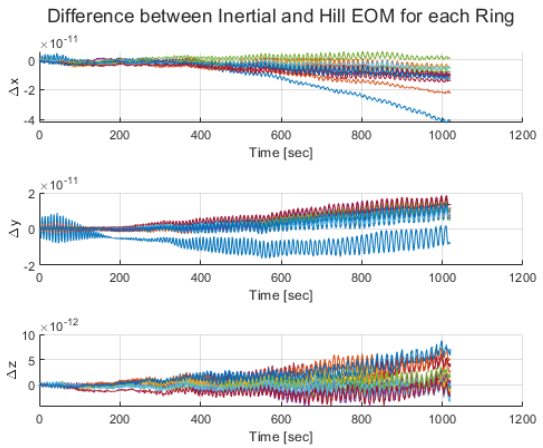

Fig. 11 Difference in Orbit frame and Inertial frame EOM integration, J_2 + drag perturbations

Figure 10 shows the worst case scenario race course evolution, namely, all orbit perturbations at play in addition to nonlinear relative motion. We can observe egregious drift in almost every direction, particularly in the radial direction where the start ring drifts towards Earth by nearly 12 km. Whereas there was a chance that applying J_2 to the chief orbit might make the drift from J_2 better, the presence of drag complicates things just enough that the drift will still be rather large.

Figure 11 looks nearly identical to figure 9, which implies that most of the error in this system comes from the drag implementation. However, the order is still on the scale of nanometers, so the accuracy is sufficient regardless of the perturbations applied.

VII. Conclusion

To conclude, the assumption made in [1] that relative motion of the rings could be ignored is resoundingly false. Even in the absence of any orbit perturbations, the relative geometry shifts by almost 1.5 km in both radial directions from the chief when relative EOM are applied. Adding J_2 results in an even more dramatic shift, and including drag exacerbates the drift even more.

Another thing to consider is that these scenarios were only run for 17 minutes, which is the average duration of a CubeSat race determined in [1]. If we propagate for a full chief's orbit with all perturbations and relative motion, then we get Figure 12.

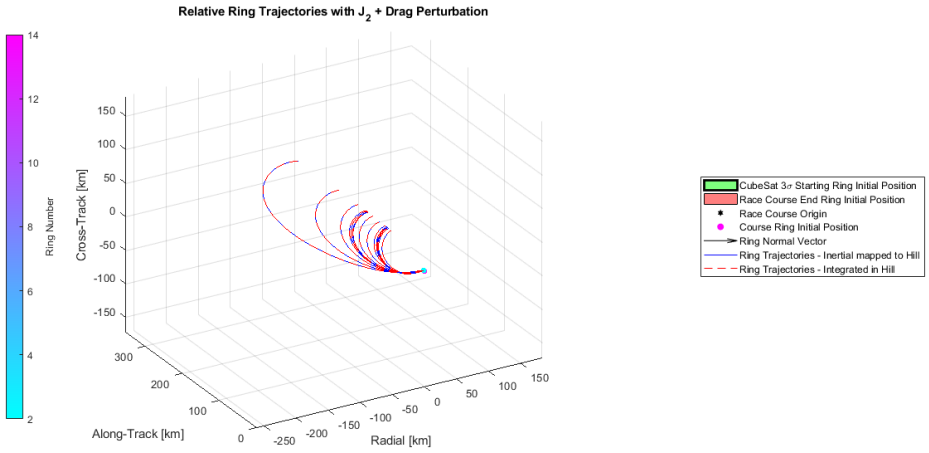

Fig. 12 Race Course Geometry over 1 Chief Orbit

On this time scale, the race course stops appearing like a race course and instead becomes a lightly correlated mass of debris. This implies that the solution isn't to simply control the race course formation at all times, as that will be expensive and severely shorten the lifespan of the race course. Instead, a future project might investigate a "deployment/stow" sequence, in which rings are all initially arrayed in a lead-follower formation, then deployed to their randomized positions before a race starts with full formation control, then stowed upon the completion of the race course to the same lead-follower formation. In addition, applying J_2 to the course origin could be investigated as a way to save fuel when maintaining the race course geometry, as there isn't anything special about ensuring the course origin is a perfect Keplerian orbit.

VIII. Acknowledgments

Special thanks to Dr. Scheeres for initially approving this deceptively fascinating project in the Optimal Trajectories course at CU Boulder, as well as to Dr. Schaub for approving me to continue to investigate it in the Spacecraft Formation Flying course at CU Boulder.

References

[1] Faber, I., "NASCAR for Cubesats: An Application of Primer Vector Theory," , 2025. URL `https://github.com/EpicIan60142/NASCARforCubeSats`.

[2] Lawden, D. F., *Analytical Methods of Optimization*, 1st ed., Dover Publications Inc., 2006.

[3] Schaub, H., and Junkins, J. L., *Analytical Mechanics of Space Systems*, 4th ed., AIAA, 2018.

[4] Yu, P., Lu, R., He, W., and Li, L. K., "Steady flow around an inclined torus at low Reynolds numbers: Lift and drag coefficients," *Computers Fluids*, Vol. 171, 2018, pp. 53–64. https://doi.org/https://doi.org/10.1016/j.compfluid.2018.05.017, URL https://www.sciencedirect.com/science/article/pii/S0045793018302603.

A. Appendix Index

Use the appendix index below to navigate to the different appendices!

B. ode45 code for Task 3

Back to appendix index: A.A

B. Outer ode45 wrapper

Back to code index: B.A

```
function dX = multiSatOrbitEOMRelative(t, X, pConst, scConst, disturb)
% Function that integrates chief motion and multiple deputies at the same
% time
%   Inputs:
%       - t: Current integration time
%       - X: State vector of n stacked satellite states, like so:
%            [X_chief; X_deputy1; ...; X_deputyN], where
%            X_chief = [X; Y; Z; Xdot; Ydot; Zdot] in the inertial frame
%              and
%            X_deputyN = [x; y; z; xDot; yDot; zDot] in the Hill frame
%       - pConst: Planetary constants structure containing Ri, mu, and J2
%       - scConst: Vector of spacecraft parameter structures containing
%                  mass, coefficient of drag, and cross-sectional area
%       - disturb: Boolean of disturbances to include or ignore in the
%                  following order for each satellite:
%           - J2: Include (true), ignore (false)
%           - Drag: Include (true), ignore (false)
%                  disturb = [disturb_sat1, disturb_sat2, ...,
%                             disturb_satn]
%   Outputs:
%       - dX: Rate of change of the state vectors, like so:
%             [dX_1; dX_2; ...; dX_n]
%
%   By: Ian Faber, 10/07/2025
%

% Number of states being integrated
nStates = 6;

% Extract chief state
X_chief = X(1:nStates);

% Extract deputies and determine the number that exist
```

```
X_deputies = X(nStates+1:end);

nDeputies = length(X_deputies)/nStates;

% Prepare rate of change vector
dX = [];

% Calculate chief rate of change and add to overall rate of change vector
dX_chief = orbitEOMInertial(t, X_chief, pConst, scConst(1), disturb(:,1));

dX = [dX; dX_chief];

% Calculate intermediate quantities
    % Calculate DCM from inertial to Hill Frame
rVec = X_chief(1:3);
vVec = X_chief(4:6);

i_r = rVec/norm(rVec);
i_h = (cross(rVec, vVec)/norm(cross(rVec,vVec)));
i_theta = cross(i_h, i_r);

HN = [i_r'; i_theta'; i_h'];

    % Calculate chief radius and its rate of change
v_c_H = HN*vVec;
r_c = norm(rVec);
rDot_c = dot(v_c_H, [1;0;0]);

    % Calculate rate of change of true anomaly
h = norm(cross(rVec, vVec));
fDot = h/(r_c^2);

    % Compile into a structure
intQuant.r_c = r_c;
intQuant.rDot_c = rDot_c;
intQuant.fDot = fDot;
intQuant.HN = HN;

% Calculate EOM for each satellite
for k = 1:nDeputies
    idx = (6*k + 1):(6*k + 6);
    X_i = X(idx);
    dX_i = orbitEOMRelative(t, X_i, X_chief, intQuant, pConst, scConst(k+1),
        disturb(:,k+1));
    dX = [dX; dX_i];
end

end
```

C. Chief inertial EOM

Back to code index: B.A

```
function dX = orbitEOMInertial(t, X, pConst, scConst, disturb)
% Function for the equations of motion for a satellite in orbit around
% earth, with the option of enabling J2 perturbations
%   Inputs:
%       - t: Current integration time in sec
%       - X: State vector arranged as follows:
%            [X; Y; Z; Xdot; Ydot; Zdot]
%       - pConst: Planetary constant vector containing mu, Ri, and J2
%       - scConst: Spacecraft parameter structure containing mass,
%                  coefficient of drag, and cross-sectional area
%       - disturb: Boolean of disturbances to include or ignore in the
%                  following order:
%           - J2: Include (true), ignore (false)
%           - Drag: Include (true), ignore (false)
%   Outputs:
%       - dX: State rate of change vector as follows:
%             [Xdot; Ydot; Zdot; Xddot; Yddot; Zddot]
%
%   By: Ian Faber, 08/26/2025
%

% Extract states from state vector
rVec = X(1:3);
r = norm(rVec);

vVec = X(4:6);

% Extract disturbance booleans
J2_enable = disturb(1);
if length(disturb) > 1
    drag_enable = disturb(2);
else
    drag_enable = false;
end

% Calculate unperturbed acceleration
a_unperturb = -(pConst.mu/(r^3))*rVec;
a_perturb = zeros(3,1);

if J2_enable
    A_J2 = [
                (15/2)*(rVec(3)/r)^2-(3/2)      0
                                                0;
                0                               (15/2)*(rVec(3)/r)^2-(3/2)
                       0;
                0                               0
                                                (15/2)*(rVec(3)/r)^2-(9/2)
           ];

    a_perturb = a_perturb + ((pConst.Ri/r)^2)*pConst.J2*A_J2*(pConst.mu/(r^3))
       *rVec;
```

```
end

if drag_enable
    v = norm(vVec);
    rho = calcDensity(pConst, r);

    a_perturb = a_perturb - 0.5*rho*(scConst.Cd*scConst.A/scConst.m)*v*vVec;
end

accel = a_unperturb + a_perturb;

dX = [vVec; accel];

end
```

D. Deputy relative EOM

Back to code index: B.A

```
function dX = orbitEOMRelative(t, X, X_c, intQuant, pConst, scConst, disturb)
% Function for the equations of motion for a deputy in a formation with a
% chief satellite in the presence of perturbations
%   Inputs:
%       - t: Current integration time in sec
%       - X: Deputy state vector arranged as follows:
%            [x; y; z; xDot; yDot; zDot]
%       - X_c: Chief state vector arranged as follows:
%              [X; Y; Z; Xdot; Ydot; Zdot]
%       - intQuant: Structure of intermediate quantities with the following
%                   fields:
%                   - r_c: Chief orbit radius at this instant of time
%                   - rDot_c: Chief orbit radius rate of change
%                   - fDot: True anomaly rate of change
%                   - HN: DCM to the chief's hill frame from the inertial
%                         frame
%       - pConst: Planetary constant vector containing mu, Ri, J2, and
%                 exponential atmosphere model parameters
%       - scConst: Spacecraft parameter structure containing mass,
%                  coefficient of drag, and cross-sectional area
%       - disturb: Boolean of disturbances to include or ignore in the
%                  following order:
%           - J2: Include (true), ignore (false)
%           - Drag: Include (true), ignore (false)
%   Outputs:
%       - dX: State rate of change vector as follows:
%             [xDot; yDot; zDot; xDDot; yDDot; zDDot]
%
%   By: Ian Faber, 08/26/2025
%

% Extract states from state vectors
x = X(1);
y = X(2);
z = X(3);
xDot = X(4);
yDot = X(5);
zDot = X(6);

X_d_N = convDeputyH2N(X_c, X, pConst);

rVec_d = X_d_N(1:3);
vVec_d = X_d_N(4:6);

cart.rVec = rVec_d;
cart.vVec = vVec_d;
cart.mu = pConst.mu;
oe_d = convCart2ClassicOE(cart);

% Extract intermediate quantities
r_c = intQuant.r_c;
```

```
rDot_c = intQuant.rDot_c;
fDot = intQuant.fDot;
HN = intQuant.HN;

% Calculate intermediate quantities
    % DCM to deputy hill frame from inertial frame
r_d = norm(rVec_d);
v_d = norm(vVec_d);

hVec_d = cross(rVec_d, vVec_d);
h_d = norm(hVec_d);

i_r_d = rVec_d/r_d;
i_h_d = hVec_d/h_d;
i_theta_d = cross(i_h_d, i_r_d);

HdN = [i_r_d'; i_theta_d'; i_h_d'];

    % DCM to chief hill frame from deputy hill frame
HHd = HN*HdN';

    % Deputy eccentricity
e_d = oe_d.e;

    % Deputy true anomaly
f_d = oe_d.f;

    % Deputy semi-latus rectum
p_d = oe_d.a*(1-e_d^2);

% Extract disturbance booleans
J2_enable = disturb(1);
if length(disturb) > 1
    drag_enable = disturb(2);
else
    drag_enable = false;
end

% Calculate unperturbed acceleration
xDDot = 2*fDot*(yDot - y*(rDot_c/r_c)) + x*fDot^2 + pConst.mu/(r_c^2) - (
    pConst.mu/(r_d^3))*(r_c + x);
yDDot = -2*fDot*(xDot - x*(rDot_c/r_c)) + y*fDot^2 - (pConst.mu/(r_d^3))*y;
zDDot = -(pConst.mu/(r_d^3))*z;

a_unperturb = [xDDot; yDDot; zDDot];
a_perturb = zeros(3,1);

if J2_enable
    A_J2 = [
                (15/2)*(rVec_d(3)/r_d)^2-(3/2)      0
                                                    0;
                0                                   (15/2)*(rVec_d(3)/r_d)^2-(3/2)
                        0;
                0                                   0
```

```
                                                    (15/2)*(rVec_d(3)/r_d)
                    ^2-(9/2)
            ];

    a_perturb = a_perturb + ((pConst.Ri/r_d)^2)*(pConst.mu/(r_d^3))*pConst.J2*
        HN*A_J2*rVec_d;
end

if drag_enable
    rho = calcDensity(pConst, r_d);

    a_perturb = a_perturb - 0.5*rho*(scConst.Cd*scConst.A/scConst.m)*v_d*HHd*[
        h_d*e_d*sin(f_d)/p_d; h_d/r_d; 0];
end

accel = a_unperturb + a_perturb;

dX = [xDot; yDot; zDot; accel];

end
```

C. Scenario Animations

Back to appendix index: A.A

To access animations for each Scenario in this report, click the associated Google Drive link!

- Scenario 1 - no perturbations: https://drive.google.com/file/d/154DiaiHJ57dG4hFhNS_68RxXZJ_9cu8C/view?usp=sharing
- Scenario 2 - J_2 perturbation: https://drive.google.com/file/d/12_ujpxLEmPYXJSR5mMQ3_IfFbX_RkHJl/view?usp=sharing
- Scenario 3 - drag perturbation: https://drive.google.com/file/d/1tu_zN6MbZHmc_2DS22otoeQmDxFLcAWV/view?usp=sharing
- Scenario 4 - J_2 + drag perturbations: https://drive.google.com/file/d/1xto6TIupmOh0-fje_1Jx__UdvjQ9mbQN/view?usp=sharing

Enjoy!